

Copilot Médico: extensão e sistema gerenciador de pacientes

Cliente: Dr. Rui

Equipe:
Aline Ferreira Barbosa
Italo Rangel
Aimê Carvalho Guimarães
Camila de Sá Vaz
Eryck Emmanuels
Lays Leticia
Leticia Rodrigues

Sumário

1. Introdução....4
2. Requisitos....4
a. Fluxo de informação....4
Etapa 1 – Início da Consulta....5
b. Camada apresentação....6
c. Camada aplicação....7
3. Tecnologias utilizadas....8
a. Backend....8
b. Frontend....8
c. Github - Repositório....8
4. Passo a passo para configuração e execução do sistema....8
5. Funcionalidades implementadas....10
6. Backlog....14

1. Introdução

Este projeto tem como objetivo o desenvolvimento de um sistema gerenciador que atuará como suporte à extensão do Copilot Médico, oferecendo recursos para gravação, processamento e transcrição de áudios de consultas, com o propósito de tornar o atendimento médico mais ágil, eficiente e organizado.

O sistema gerenciador disponibiliza um conjunto de funcionalidades que apoiam a gestão das informações geradas durante as consultas, contribuindo para a otimização do fluxo de trabalho dos profissionais de saúde e para a melhoria da documentação clínica.

O sistema gerenciador possui as seguintes funcionalidades:
Gestão de pacientes/Prontuário: rastreio das consultas realizadas via extensão - armazenando dos áudios e transcrições;
Exame: extração de informações dos exames;
Dashboard: principais indicadores de número de pacientes, consultas realizadas, etc;

2. Requisitos

a. Fluxo de informação

O fluxo de informação é iniciado pelo profissional de saúde no Sistema Gerenciador Web, seguindo uma arquitetura cliente-servidor, na qual o sistema gerenciador atua como elemento central de orquestração entre a extensão Copilot Médico, os serviços de processamento e os módulos de Inteligência Artificial, conforme a imagem a seguir:

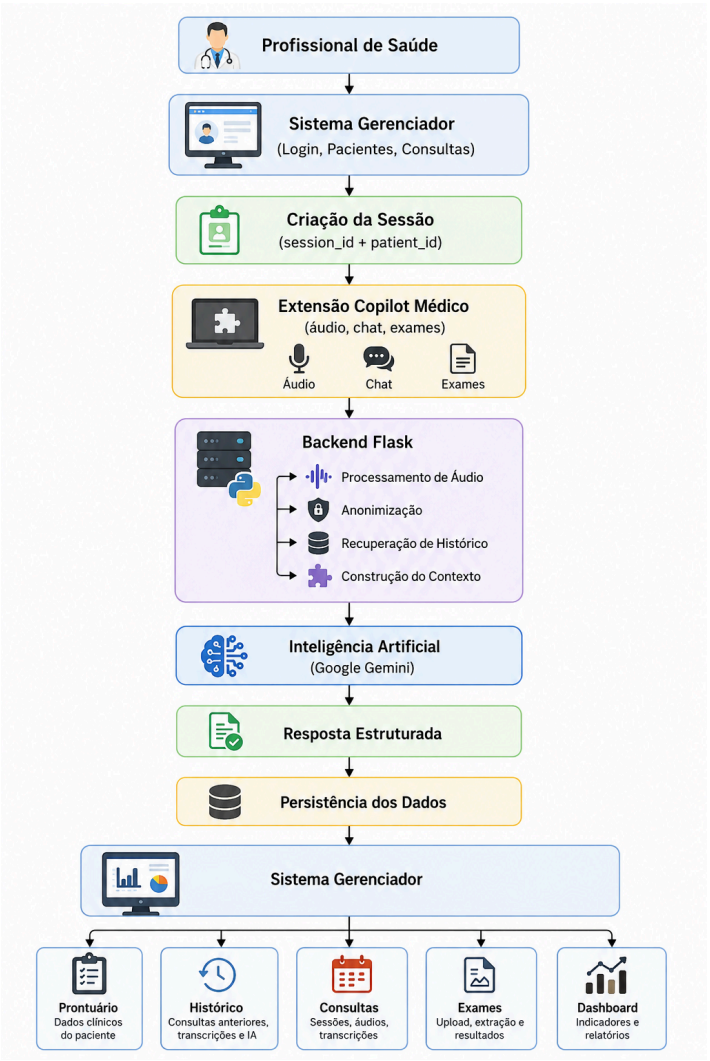


Imagem 1: Fluxo da informação

Etapa 1 – Início da Consulta

1. O profissional de saúde autentica-se no Sistema Gerenciador.
2. O usuário seleciona um paciente existente ou realiza um novo cadastro.
3. O Sistema Gerenciador cria uma nova sessão de consulta, gerando um identificador único (session_id) associado ao paciente (patient_id) e ao profissional responsável.

Etapa 2 – Integração com a Extensão

4. O Sistema Gerenciador envia as informações da sessão para a Extensão Copilot Médico, permitindo que a consulta seja iniciada.
5. Durante a consulta, o usuário pode:

- iniciar ou interromper a gravação de áudio;
- enviar mensagens ao assistente;
- realizar upload de exames;

- solicitar extração de informações clínicas;
- consultar sugestões geradas pela IA.

Etapa 3 – Envio das Informações

6. A Extensão captura as ações realizadas pelo usuário e envia requisições ao Backend (Flask), juntamente com:

- identificador da consulta (session_id);
- identificador do paciente (patient_id);
- dados capturados durante a consulta.

Etapa 4 – Processamento Inicial

7. O Backend recebe a requisição e realiza o processamento inicial dos dados.
8. Quando necessário, os dados passam por mecanismos de anonimização e remoção de informações sensíveis utilizando o módulo responsável pelo tratamento de texto.

Etapa 5 – Recuperação do Contexto Clínico

9. O Sistema Gerenciador recupera o histórico clínico do paciente, incluindo:

- consultas anteriores;
- transcrições;
- exames;
- diagnósticos prévios;
- demais informações disponíveis no prontuário.

10. Todas essas informações são utilizadas para compor o contexto clínico da consulta atual.

b. Camada apresentação

A primeira camada de apresentação é implementada pelo **Sistema Gerenciador Web**, responsável por fornecer interfaces para autenticação, gerenciamento de pacientes, consultas, prontuários, exames e dashboards gerenciais. Essa camada permite que os profissionais de saúde realizem atividades administrativas e acompanhem as informações geradas durante os atendimentos.

A segunda camada de apresentação é implementada pela **Extensão Copilot Médico**, desenvolvida para operar no navegador Google Chrome e atuar diretamente durante a consulta médica. Essa extensão fornece recursos de apoio ao atendimento, tais como gravação e transcrição de áudio, interação com o assistente inteligente, upload de exames e visualização de sugestões produzidas pela Inteligência Artificial em tempo real.

c. Camada aplicação

A camada de aplicação (Backend) é responsável pelo processamento das requisições provenientes tanto do Sistema Gerenciador quanto da Extensão, implementando as regras de negócio da solução. Essa camada realiza o gerenciamento das sessões de consulta, recuperação do histórico clínico, anonimização de dados sensíveis, processamento de documentos e integração com os serviços de Inteligência Artificial.

O fluxo de informações foi projetado seguindo uma arquitetura cliente-servidor, na qual o Sistema Gerenciador atua como núcleo central da solução, sendo responsável pela persistência e gerenciamento dos dados clínicos, enquanto a extensão opera como um módulo de apoio operacional durante as consultas.

3. Tecnologias utilizadas

a. Backend

Backend
├── Python
├── Flask
├── Flask-CORS
├── RabbitMQ
├── spaCy
├── Vosk
├── Gemini API
├── PyPDF2
└── Docker

b. Frontend

Frontend Web
├── React
├── HTML5
├── CSS3
├── Dashboard Libraries
└── Fetch/Axios

Frontend Extensão
├── React
├── JavaScript
├── Chrome Extension API
├── Web Audio API
└── Fetch API

c. Github - Repositório

https://github.com/qleticia/copilot_medico_grupo_2_es

4. Passo a passo para configuração e execução do sistema

Aplicação React/Vite da interface web do Copilot Médico.

A aplicação consome o backend Flask compartilhado em `API/`. A base de pacientes não é separada: a listagem usa `GET /api/all-patients` e o cadastro usa `POST /api/patients`.

Variáveis de ambiente

Crie um arquivo `.env` nesta pasta quando precisar alterar a URL da API:

```
VITE_API_URL=http://localhost:3001
```

Se a variável não for informada, a web usa `http://localhost:3001`.

Como rodar

Em um terminal, suba o backend:

```
cd API
python server.py
```

Em outro terminal, rodei uma web:

```
cd front/copmed-web
npm install
npm run dev
```

Por padrão, o Vite abre a web em `http://localhost:5174`.

Funcionalidades nesta etapa:

- Login através de `POST /api/auth/login`.
- Armazenamento do token Bearer em `localStorage`.
- Navegação básica: Dashboard, Pacientes, Agendamentos, Análise com IA, Atendimentos, Relatórios e Configurações.
- Lista de pacientes via backend compartilhado.
- Cadastro de paciente via backend compartilhado.
- Visualização inicial de consultas do paciente via `GET /api/patients/:id/consultations`.

5. Funcionalidades implementadas

US01 – Realizar Login

Como profissional de saúde,
quero autenticar-me no sistema,
para que eu possa acessar apenas as funcionalidades autorizadas.

Critérios de Aceitação

- O sistema deve permitir informar usuário e senha;
- O sistema deve validar as credenciais informadas;
- O sistema deve iniciar uma sessão autenticada;
- O sistema deve impedir o acesso em caso de credenciais inválidas.

US02 – Cadastrar Paciente

Como profissional de saúde,
quero cadastrar novos pacientes,
para que suas informações possam ser utilizadas durante as consultas.

Critérios de Aceitação

- O sistema deve permitir o cadastro de dados do paciente;
- O paciente deve receber um identificador único;
- As informações devem ser armazenadas para consultas futuras.

US03 – Consultar Pacientes

Como profissional de saúde,
quero pesquisar pacientes cadastrados,
para que eu possa acessar rapidamente suas informações.

Critérios de Aceitação

- O sistema deve permitir pesquisa pelo nome do paciente;
- O sistema deve exibir a lista de pacientes encontrados;
- O usuário deve conseguir selecionar um paciente para visualizar seus dados.

US04 – Iniciar Consulta

Como profissional de saúde,
quero iniciar uma nova consulta para um paciente,
para que as informações do atendimento sejam registradas.

Critérios de Aceitação

- O sistema deve criar uma nova sessão de consulta;
- A consulta deve ser associada ao paciente selecionado;
- Deve ser gerado um identificador da consulta.

US05 – Encerrar Consulta

Como profissional de saúde,
quero finalizar uma consulta,
para que os dados gerados sejam armazenados no histórico do paciente.

Critérios de Aceitação

- O sistema deve registrar a data e horário de encerramento;
- Todas as informações produzidas durante a consulta devem ser persistidas.

US06 – Visualizar Prontuário

Como profissional de saúde,
quero acessar o prontuário do paciente,
para que eu possa consultar informações clínicas anteriores.

Critérios de Aceitação

- O sistema deve exibir consultas anteriores;
- O sistema deve exibir exames associados ao paciente;
- O sistema deve exibir transcrições e informações produzidas pela IA.

US07 – Recuperar Histórico Clínico

Como profissional de saúde,
quero visualizar o histórico clínico do paciente,
para que eu possa realizar atendimentos mais contextualizados.

Critérios de Aceitação

- O sistema deve recuperar automaticamente consultas anteriores;

- O sistema deve recuperar exames previamente cadastrados;
- O histórico deve ser utilizado para contextualizar as respostas da IA.

US08 – Armazenar Informações Clínicas

Como profissional de saúde,
quero que todas as informações produzidas durante a consulta sejam armazenadas,
para que possam ser recuperadas posteriormente.

Critérios de Aceitação

- O sistema deve armazenar interações realizadas;
- O sistema deve armazenar transcrições;
- O sistema deve armazenar respostas produzidas pela IA;
- O sistema deve armazenar informações extraídas dos exames;
- Os dados devem permanecer associados ao paciente.

US09 – Consultar Assistente Inteligente

Como profissional de saúde,
quero enviar perguntas ao assistente baseado em IA,
para que eu obtenha apoio durante a consulta.

Critérios de Aceitação

- O sistema deve enviar a pergunta para o serviço de IA;
- O sistema deve considerar o histórico do paciente;
- O sistema deve apresentar a resposta ao usuário.

US10 – Gerar Informações Clínicas Estruturadas

Como profissional de saúde,
quero que a IA interprete os dados da consulta,
para que informações relevantes sejam identificadas automaticamente.

Critérios de Aceitação

- O sistema deve processar a transcrição da consulta;
- O sistema deve gerar respostas contextualizadas;
- O sistema deve armazenar as respostas produzidas.

US11 – Realizar Upload de Exames

Como profissional de saúde,
quero anexar exames do paciente,
para que essas informações possam ser utilizadas durante o atendimento.

Critérios de Aceitação

- O sistema deve permitir upload de arquivos PDF;
- O exame deve ser associado ao paciente;
- O documento deve ser armazenado no sistema.

US12 – Extrair Informações de Exames

Como profissional de saúde,
quero extrair automaticamente informações de exames,
para que eu possa analisá-las de maneira mais eficiente.

Critérios de Aceitação

- O sistema deve realizar a leitura do documento enviado;
- O sistema deve enviar o conteúdo extraído para a IA;
- O sistema deve apresentar as informações estruturadas ao usuário;
- Os dados extraídos devem ser persistidos no histórico do paciente.